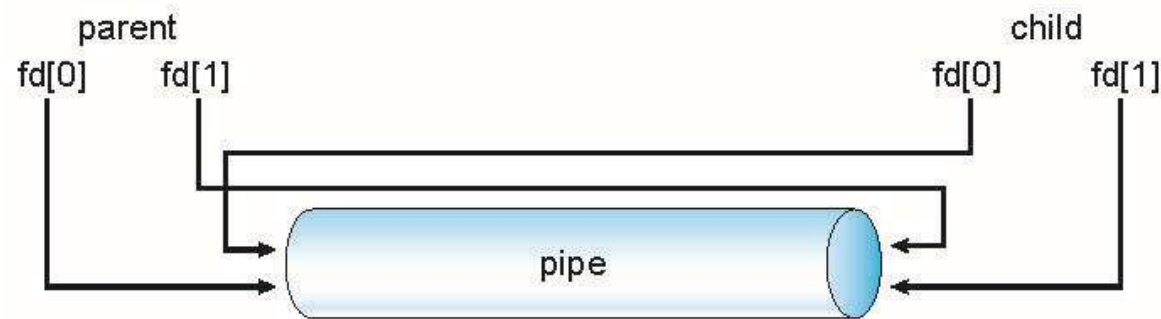


IPC

Pipe, FIFO, MMAP

Pipe Ordinarie

- Pipe ordinarie comunicano in modalità produttore-consumatore
- Produttore scrive su un'estremità (write-end della pipe)
- Consumatore legge dall'altra estremità (read-end della pipe)
- Le pipe ordinarie sono unidirezionali
- Richiedono una relazione parentale tra i processi che comunicano



Pipe Ordinarie

- All'inizio una pipe è vuota
- write aggiunge dati alla pipe
- read legge e *rimuove* dati dalla pipe
 - non si possono leggere piu' volte gli stessi dati da una pipe
 - non si puo' chiamare lseek su una pipe
 - i dati si ottengono in ordine First In First Out
- una pipe con una estremita' chiusa si dice rotta (broken)

Pipe Ordinarie

```
/* pipe1: invio di dati da un genitore ad un figlio */

#include <stdio.h>
#include <unistd.h>
#define MAXLINE 64

int main(void)
{
    int n, fd[2];
    pid_t pid;
    char line[MAXLINE];

    if (pipe(fd) < 0) perror("pipe"), exit(1);

    if ( (pid = fork()) < 0) perror("fork"), exit(1);

    else if (pid > 0) { /* genitore */
        close(fd[0]);
        write(fd[1], "hello world\n", 12);
    }
    else { /* figlio */
        close(fd[1]);
        n = read(fd[0], line, MAXLINE);
        write(STDOUT_FILENO, line, n);
    }
    exit(0);
}
```

Pipe Ordinarie

- Occorre chiudere le imboccature non utilizzate

```
int main(void) {
    int fd[2];
    char buf[64];
    pid_t pid;

    pipe(fd);

    if ((pid = fork()) == 0) {    // Figlio lettore
        // ERRORE: non chiude fd[1] (scrittura)
        // close(fd[1]);    // <- manca di proposito
        while (read(fd[0], buf, sizeof(buf)) > 0)
            write(STDOUT_FILENO, buf, 5);
        close(fd[0]);
        _exit(0);
    } else {                    // Padre scrittore
        close(fd[0]);           // chiude lettura
        write(fd[1], "ciao\n", 5);
        close(fd[1]);           // segnala EOF (ma figlio tiene ancora aperto fd[1])
        printf("Padre: attendo il figlio...\n");
        wait(NULL);             // <-- si blocca per sempre!
        printf("Padre: figlio terminato.\n");
    }
}
```

Pipe con Nome

- **Named Pipes (FIFO)** sono più potenti delle pipe ordinarie
 - Non è necessaria una relazione parentale tra processi comunicanti
 - Molti processi possono usare le named pipe per comunicare
-
- Creato un file speciale con **makefifo(nome,modo)** nel FS
 - Ha un **i-node** e può essere aperto in lettura/scrittura
 - Deve essere cancellato con **unlink**
 - Si usa in modalità unidirezionale (anche se può essere aperta RW)

Pipe con Nome

```
#include <stdio.h>
#include <errno.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#define MAX_BUF_SIZE 1000

/* Convert to the string to upper case */
count = 0;
while (count < numread) {
    buf[count] = toupper(buf[count]);
    count++;
}
printf("Server : Converted String : %s\n", buf);
}

int main(int argc, char *argv[]){

int fd, ret_val, count, numread;
char buf[MAX_BUF_SIZE];

/* Create the named - pipe */
ret_val = mkfifo("miafifo", 0666);
if ((ret_val == -1) && (errno != EEXIST)) {
    perror("Error creating the named pipe");
    exit (1); }

/* Open the pipe for reading */
fd = open("miafifo", O_RDONLY);

/* Read from the pipe */
numread = read(fd, buf, MAX_BUF_SIZE);
buf[numread] = '\0';
printf("Server : Read From the pipe : %s\n", buf);
```

Codice Server

Pipe con Nome

```
#include <stdio.h>
#include <errno.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#define MAX_BUF_SIZE 1000

/* Convert to the string to upper case */
count = 0;
while (count < numread) {
    buf[count] = toupper(buf[count]);
    count++;
}
printf("Server : Converted String : %s\n", buf);
}

int main(int argc, char *argv[]){

int fd, ret_val, count, numread;
char buf[MAX_BUF_SIZE];

/* Create the named - pipe */
ret_val = mkfifo("miafifo", 0666);
if ((ret_val == -1) && (errno != EEXIST)) {
    perror("Error creating the named pipe");
    exit (1); }

/* Open the pipe for reading */
fd = open("miafifo", O_RDONLY);

/* Read from the pipe */
numread = read(fd, buf, MAX_BUF_SIZE);
buf[numread] = '\0';
printf("Server : Read From the pipe : %s\n", buf);
```

Codice Server

Pipe con Nome

```
#include <stdio.h>
#include <errno.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

Codice Client

```
int main(int argc, char *argv[])
{
    int fd;

    /* Check if an argument was specified. */

    if (argc != 2) {
        printf("Usage : %s <string to be sent to the server>n", argv[0]);
        exit (1);
    }

    /* Open the pipe for writing */
    fd = open("miafifo", O_WRONLY);

    /* Write to the pipe */
    write(fd, argv[1], strlen(argv[1]));
}
```

Esempio esecuzione:

- `./servFifo &`
- `./clientFifo prova`
- Server : Read From the pipe : prova
- Server : Converted String : PROVA

Pipe con Nome

Operazione

```
open("fifo", O_RDONLY)
```

```
open("fifo", O_WRONLY)
```

```
open("fifo", O_RDWR)
```

Cosa fa il kernel

Blocca finché non c'è **almeno un writer** aperto

Blocca finché non c'è **almeno un reader** aperto

Non blocca mai (apre entrambe le direzioni nello stesso processo, ma è raro/ambiguo)

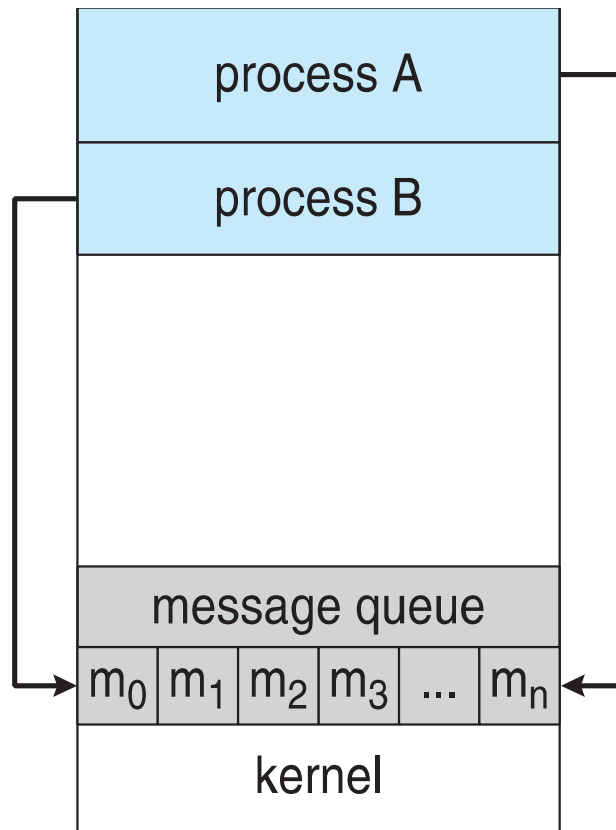
- Se apre con O_WRONLY | O_NONBLOCK, il writer **non si blocca**, se senza lettori riceve subito errore ENXIO
- Se apre con O_RDONLY | O_NONBLOCK, il reader **non si blocca**, ma restituirà subito 0 (EOF) finché non arriva un writer

Memoria Condivisa

- Comunicazione su regione di **shared memory**
- Tipicamente creata nello spazio di indirizzamento del processo che la crea (segmento di shared memory)
- Gli altri processi devono accedere a questo spazio che generalmente è proibito (va rilassata la restrizione)
- I processi leggono e scrivono su questo spazio che è gestito dai processi (modalità utente)
- La coordinazione di lettura e scrittura è sotto la responsabilità dei processi coinvolti, il SO lascia fare

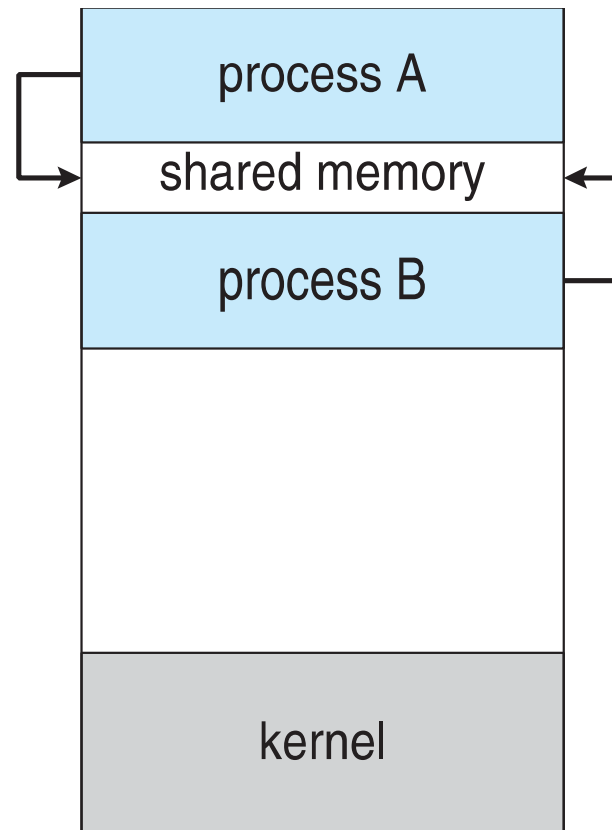
Memoria Condivisa vs Pipe

(a) Message passing. (b) shared memory.



(a)

Scambio di poca informazione, tutti gli scambi passano per il Kernel



(b)

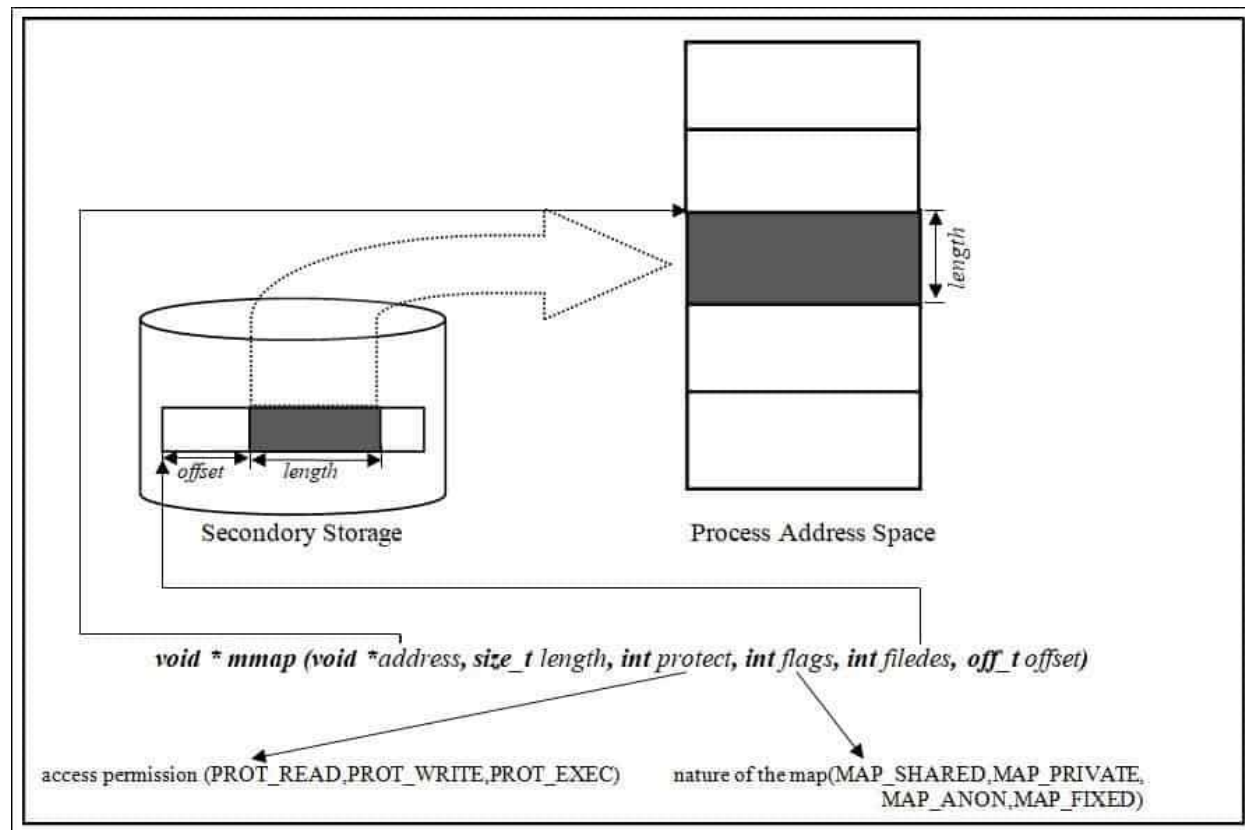
Più veloce (kernel alloca la zona condivisa), più semplice accesso, problema coordinazione tra processi

mmap

- La funzione **mmap**: mappa in process address space un file o un device
 - Una volta mappato accessibile in modo diretto

`void * mmap (void *address, size_t length, int protect, int flags, int fildes, off_t offset)`

- **address**: indirizzo di partenza
- **length**: bytes richiesti
- **protect**:
PROT_READ | PROT_WRITE |
PROT_EXEC | PROT_NONE
- **flags**:
MAP_SHARED, MAP_PRIVATE
MAP_ANON, MAP_FIXED
- **fildes**: descrittore di file
- **offset**: offset del file



Produttore

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* strings written to shared memory */
    const char *message_0 = "Hello";
    const char *message_1 = "World!";

    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* write to the shared memory object */
    sprintf(ptr, "%s", message_0);
    ptr += strlen(message_0);
    sprintf(ptr, "%s", message_1);
    ptr += strlen(message_1);

    return 0;
}
```

Aperta con nome "OS" in modalità R/W - se non esiste creata con permessi R/W -

Dimensione SIZE

Mappata in memoria in modalità MAP_SHARED - cambiamenti visibili a chi condivide la memoria

PROT_WRITE permette la scrittura

Puntatore ptr incrementato in modalità non automatica

Consumatore

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s", (char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```

Aperta con nome "OS" in
modalità lettura con permessi
R/W

Dimensione SIZE

Mappata in memoria in modalità
MAP_SHARED - cambiamenti
visibili a chi condivide la
memoria

PROT_READ permette la lettura

Rimuove l'oggetto shared mem

Mmap Anonima

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <unistd.h>
#include <string.h>

int main(void) {
    size_t SIZE = 256;
    char *shared = mmap(NULL, SIZE,
                        PROT_READ | PROT_WRITE,
                        MAP_SHARED |
MAP_ANONYMOUS,
                        -1, 0);
    if (shared == MAP_FAILED) {
        perror("mmap");
        return 1;
    }
}
```

```
pid_t pid = fork();
if (pid == -1) { perror("fork"); return 1; }

if (pid == 0) {
    // FIGLIO: scrive nel buffer
    strcpy(shared, "Ciao dal figlio via mmap anonima!");
    _exit(0);
}

// PADRE: aspetta e poi legge
waitpid(pid, NULL, 0);
printf("[Padre] Letto: \"%s\"\n", shared);

munmap(shared, SIZE);
return 0;
}
```